BAHAN AJAR
INFORMATIKA

PEMBELAJARAN MANDIRI - SIKLUS 4 - PERTEMUAN 11 ~ 13

UNTUK SISWA SMK KELAS 10



DISUSUN OLEH: IKHSAN QADRI, S.PD.

Bahan Ajar Informatika: Konsep Dasar Struktur Data

Array, Stack (LIFO), dan Queue (FIFO) dalam Pemrograman · Elemen Berpikir Komputasional · Fase E / Kelas X SMK · Pertemuan 11

Disusun oleh: Ikhsan Qadri, S.Pd.

Peta Belajar

Sampai Pertemuan 10 kamu mengolah data dengan tabel, rumus, dan grafik. Mulai hari ini kita masuk ke dunia **pemrograman**: bagaimana data **disimpan di dalam program** agar bisa diambil dengan cara yang tepat? Setelah membaca dan berlatih, kamu diharapkan mampu:

1. Menjelaskan konsep **array**, **stack (LIFO)**, dan **queue (FIFO)**.
2. Mensimulasikan operasi **push/pop** dan **enqueue/dequeue** serta menentukan indeks data.
3. Memilih struktur data yang sesuai untuk skenario tertentu (misalnya antrean cetak dan fitur *Undo*) beserta alasannya.
4. Membandingkan secara singkat **array dan linked list** (pengayaan).

Pertanyaan pemantik untuk dibawa selama membaca:

- Struktur data mana yang paling efisien untuk antrean cetak?
- Jika membuat fitur “Undo”, metode penyimpanan apa yang cocok?
- Apa beda stack dan queue?
- Bagaimana memanggil data pada urutan tertentu dalam array?

> ***Pemahaman kunci:*** *cara data disimpan menentukan cara data dapat diakses. Array, stack, dan queue memiliki perilaku berbeda yang cocok untuk kebutuhan berbeda.*

1. Mengapa Cara Menyimpan Data Itu Penting?

Bayangkan tiga situasi sehari-hari:

- **Tumpukan piring** di kantin. Piring yang kamu ambil adalah yang **terakhir** diletakkan di atas tumpukan.
- **Antrean di kasir.** Orang yang dilayani lebih dulu adalah yang **pertama** datang.
- **Rak berlaci bernomor.** Kamu bisa langsung membuka laci nomor 5 tanpa membuka laci lain.

Tiga cara menyimpan itu menghasilkan tiga aturan pengambilan yang berbeda. Dalam pemrograman pun demikian. Data yang sama bisa disimpan dengan cara berbeda, dan cara itu menentukan **data mana yang keluar lebih dulu**, **seberapa cepat** data ditemukan, dan **seberapa mudah** program diubah.

Struktur data adalah cara mengatur dan menyimpan data di dalam program agar dapat diakses dan diolah secara efisien.

Analogi	Struktur data	Aturan
Rak berlaci bernomor	**Array**	Ambil lewat nomor (indeks)
Tumpukan piring/buku	**Stack**	Terakhir masuk, pertama keluar (LIFO)
Antrean kasir/kantin	**Queue**	Pertama masuk, pertama keluar (FIFO)

2. Array

Array adalah kumpulan data **berurutan** yang masing-masing punya nomor posisi yang disebut **indeks**. Dalam kebanyakan bahasa pemrograman (termasuk Python), **indeks dimulai dari 0**, bukan 1.

Contoh

Array `[A, B, C]`:

Indeks	0	1	2
Isi	A	B	C

- Elemen **indeks 0** adalah **A** (elemen pertama).
- Elemen **indeks 2** adalah **C** (elemen terakhir).
- Jika array berisi **n** elemen, indeks terakhirnya adalah **n − 1**.

Di Python, array biasa ditulis dengan **list**:

```
nilai = [80, 75, 90]
print(nilai[0])     # 80  (elemen indeks 0)
print(nilai[2])     # 90  (elemen terakhir)
nilai[1] = 78       # mengubah isi indeks 1
print(nilai)        # [80, 78, 90]
print(len(nilai))   # 3   (banyak elemen)
```

Memanggil Data pada Urutan Tertentu

Inilah jawaban pertanyaan pemantik keempat: **gunakan indeks**. Untuk memanggil data urutan ke-*k* (dihitung dari 1), pakai indeks *k − 1*. Data urutan ke-3 berada pada indeks 2.

Operasi Dasar pada Array

Operasi	Arti	Contoh (Python)
Baca	Mengambil isi pada indeks tertentu	`nilai[2]`
Ubah	Mengganti isi pada indeks tertentu	`nilai[1] = 78`
Tambah di akhir	Menambah elemen di ujung	`nilai.append(85)`
Hapus	Membuang elemen	`nilai.pop(0)` atau `del nilai[0]`

Operasi	Arti	Contoh (Python)
Panjang	Banyak elemen	`len(nilai)`

Hati-hati: mengakses indeks di luar rentang (misalnya `nilai[10]` pada array tiga elemen) menimbulkan **kesalahan** (`IndexError` di Python). Selalu pastikan indeks berada di antara 0 dan n − 1.

Kelebihan dan Keterbatasan

Kelebihan	Keterbatasan
Akses lewat indeks **sangat cepat** (langsung ke posisi)	Menyisipkan atau menghapus di **tengah** menuntut elemen lain bergeser
Sederhana dan mudah dipahami	Pada banyak bahasa, ukuran array **tetap** (list Python lebih fleksibel)
Cocok untuk data berurutan bertipe sama	Posisi (indeks) tidak memuat arti, sehingga perlu dicatat artinya

3. Stack (LIFO)

Stack (tumpukan) menyimpan data dengan aturan **LIFO**: *Last In, First Out*, yaitu **terakhir masuk, pertama keluar**. Data hanya boleh ditambah dan diambil dari **satu ujung**, yang disebut **puncak (top)**.

Operasi	Arti
`push(x)`	Menaruh data x di puncak stack
`pop()`	Mengambil dan menghapus data di puncak
`peek()` / `top`	Melihat data di puncak tanpa menghapus
`isEmpty()`	Memeriksa apakah stack kosong

Awas: melakukan `pop()` pada stack kosong adalah kesalahan (disebut *underflow*).

Menelusuri Operasi Stack

Ini urutan yang dipakai pada LKPD 11: `push(A)`, `push(B)`, `push(C)`, `pop()`, `pop()`.

Langkah	Operasi	Isi stack setelah operasi (kiri = dasar, kanan = puncak)	Data keluar
1	`push(A)`	[A]	–
2	`push(B)`	[A, B]	–
3	`push(C)`	[A, B, C]	–
4	`pop()`	[A, B]	**C**

Langkah	Operasi	Isi stack setelah operasi (kiri = dasar, kanan = puncak)	Data keluar
5	`pop()`	[A]	**B**

Data yang tersisa: A, pada **indeks 0** (dasar tumpukan). Perhatikan urutan keluarnya: C lalu B, kebalikan dari urutan masuknya.

Stack di Python

```
stack = []
stack.append("A")     # push(A)
stack.append("B")     # push(B)
stack.append("C")     # push(C)
print(stack.pop())    # pop() -> C
print(stack.pop())    # pop() -> B
print(stack)          # ['A']
```

Mengapa Stack Cocok untuk "Undo"?

Saat mengedit dokumen, setiap perubahan disimpan ke stack. Perubahan **terakhir** adalah yang pertama kali dibatalkan saat kamu menekan Undo. Ini persis perilaku LIFO.

Aksi pengguna	Isi stack riwayat (dasar → puncak)
Ketik "Halo"	[ketik "Halo"]
Tebalkan teks	[ketik "Halo", tebalkan]
Ganti warna	[ketik "Halo", tebalkan, ganti warna]
Undo	[ketik "Halo", tebalkan] (ganti warna dibatalkan)
Undo	[ketik "Halo"] (tebal dibatalkan)

Contoh Penggunaan Stack Lain

- Tombol **Back (kembali)** pada peramban.
- Pemanggilan fungsi dalam program (*call stack*).
- Memeriksa kecocokan tanda kurung dalam ekspresi.

4. Queue (FIFO)

Queue (antrean) menyimpan data dengan aturan **FIFO**: *First In, First Out*, yaitu **pertama masuk, pertama keluar**. Data ditambah di **belakang (rear)** dan diambil dari **depan (front)**.

Operasi	Arti
`enqueue(x)`	Menambah data x di belakang antrean
`dequeue()`	Mengambil dan menghapus data di depan antrean
`front`	Melihat data terdepan tanpa menghapus

Operasi	Arti
`isEmpty()`	Memeriksa apakah antrean kosong

Menelusuri Operasi Queue

Urutan: `enqueue(A)`, `enqueue(B)`, `enqueue(C)`, `dequeue()`, `dequeue()`.

Langkah	Operasi	Isi queue (kiri = depan, kanan = belakang)	Data keluar
1	`enqueue(A)`	[A]	–
2	`enqueue(B)`	[A, B]	–
3	`enqueue(C)`	[A, B, C]	–
4	`dequeue()`	[B, C]	**A**
5	`dequeue()`	[C]	**B**

Data yang tersisa: C. Urutan keluarnya A lalu B, **sama** dengan urutan masuknya.

Queue di Python

Untuk queue, gunakan `deque` dari modul `collections`, karena pengambilan dari depan cepat:

```
from collections import deque

antrean = deque()
antrean.append("A")          # enqueue(A)
antrean.append("B")          # enqueue(B)
antrean.append("C")          # enqueue(C)
print(antrean.popleft())     # dequeue() -> A
print(antrean.popleft())     # dequeue() -> B
print(list(antrean))         # ['C']
```

Catatan: `list.pop(0)` juga bisa dipakai, tetapi pada data besar lebih lambat karena semua elemen harus bergeser.

5. Stack dan Queue: Apa Bedanya?

Aspek	Stack	Queue
Prinsip	LIFO (terakhir masuk, pertama keluar)	FIFO (pertama masuk, pertama keluar)
Tempat tambah	Puncak (top)	Belakang (rear)
Tempat ambil	Puncak (top)	Depan (front)
Operasi	push dan pop	enqueue dan dequeue
Analogi	Tumpukan piring	Antrean kasir

Aspek	Stack	Queue
Cocok untuk	Undo, riwayat Back, pemeriksaan tanda kurung	Antrean cetak, antrean pelanggan, antrean tugas

Urutan Operasi yang Sama, Hasil Berbeda

LKPD 11 menanyakan: *bagaimana hasilnya jika memakai queue?* Bandingkan dengan data A, B, C yang masuk berurutan lalu dikeluarkan dua kali:

	Stack	Queue
Data keluar (berurutan)	C, B	A, B
Yang tersisa	A	C

6. Memilih Struktur Data yang Sesuai

Cara memilih: tanyakan **"data mana yang ingin kuambil lebih dulu?"** dan **"apakah aku perlu langsung menuju posisi tertentu?"**

Pertanyaan	Jika ya
Data harus diambil menurut **nomor posisi**?	**Array**
Yang **terbaru** harus diambil lebih dulu?	**Stack**
Yang **paling lama menunggu** harus diambil lebih dulu?	**Queue**

Skenario dan Pilihan

Skenario	Struktur	Alasan
Antrean cetak (*print spooler*)	**Queue**	File yang dikirim lebih dulu dicetak lebih dulu (adil, FIFO)
Fitur Undo	**Stack**	Perubahan terakhir dibatalkan lebih dulu (LIFO)
Tombol Back peramban	**Stack**	Halaman terakhir dikunjungi adalah yang pertama dikembalikan
Antrean pelanggan di kasir	**Queue**	Pelanggan yang datang pertama dilayani pertama
Daftar nilai 30 siswa yang dibuka menurut nomor absen	**Array**	Akses langsung lewat indeks
Tumpukan tugas yang harus diperiksa dari yang paling atas	**Stack**	Tugas terakhir diletakkan diperiksa pertama

Jawaban pemantik 1: untuk antrean cetak, struktur yang paling sesuai adalah **queue** karena prinsip FIFO menjamin dokumen dicetak sesuai urutan kedatangan.

Catatan lanjutan: pada sistem nyata, kadang ada dokumen yang diprioritaskan sehingga dipakai **antrean prioritas**. Itu pengembangan dari queue dan akan kamu temui di tingkat lanjut.

7. Simulasi dengan Kartu Data (Paper-Based Puzzle)

Untuk memahami struktur data, **peragakan** secara fisik:

- **Stack:** letakkan kartu di dalam kotak atau gelas. Hanya kartu paling atas yang bisa diambil.
- **Queue:** letakkan kartu pada selang kertas yang terbuka kedua ujungnya. Masuk dari satu ujung, keluar dari ujung lain.
- **Array:** letakkan kartu berurutan dan beri nomor posisi 0, 1, 2, …

Contoh Puzzle: Tiga Push dan Dua Pop

Operasi: `push(Ani)`, `push(Budi)`, `push(Cici)`, `pop()`, `pop()`.

Urutan keluar: Cici, Budi. **Sisa:** Ani (indeks 0).

Contoh Puzzle Tingkat Lanjut: Operasi Campuran

Operasi: `push/enqueue(5)`, `(8)`, lalu `pop/dequeue`, `(3)`, `(9)`, lalu `pop/dequeue` dua kali.

Langkah	Operasi	Isi stack	Keluar (stack)	Isi queue	Keluar (queue)
1	masuk 5	[5]	–	[5]	–
2	masuk 8	[5, 8]	–	[5, 8]	–
3	keluar	[5]	8	[8]	5
4	masuk 3	[5, 3]	–	[8, 3]	–
5	masuk 9	[5, 3, 9]	–	[8, 3, 9]	–
6	keluar	[5, 3]	9	[3, 9]	8
7	keluar	[5]	3	[9]	3

Urutan keluar stack: 8, 9, 3 (sisa 5). **Urutan keluar queue:** 5, 8, 3 (sisa 9).

Tips menelusuri: tulis isi struktur setelah setiap operasi, jangan melompat. Kesalahan paling umum adalah salah mengambil dari ujung yang keliru.

Mengisi Tabel LKPD

Tabel temuan LKPD 11 memuat: **langkah, operasi, isi struktur setelah operasi, dan data keluar**. Gunakan format tabel penelusuran di atas, lalu jawab: *data apa yang tersisa dan di indeks berapa?*

8. Pengayaan: Array dan Linked List

Linked list adalah struktur data yang tiap elemennya (disebut **node**) menyimpan **data** dan **penunjuk (pointer)** ke node berikutnya. Node tidak harus berdekatan di memori, berbeda dengan array.

```
[Data A | →] → [Data B | →] → [Data C | →] → selesai
```

Aspek	Array	Linked list
Akses elemen ke-*k*	**Cepat** (langsung lewat indeks)	Lambat (harus menelusuri dari awal)
Sisip/hapus di tengah	Perlu menggeser banyak elemen	Cukup mengubah penunjuk
Penggunaan memori	Hemat (data rapat)	Lebih boros (ada penunjuk)
Ukuran	Cenderung tetap	Mudah bertambah dan berkurang
Cocok untuk	Data yang sering **dibaca menurut posisi**	Data yang sering **disisipi atau dihapus**

Skenario penggunaan keduanya (contoh): daftar nama peserta yang sering dibaca menurut nomor urut cocok memakai array. Daftar putar lagu yang sering disisipi lagu baru di tengah cocok memakai linked list. Banyak bahasa menyediakan struktur siap pakai sehingga kamu tidak selalu membuatnya sendiri.

9. Kesalahan dan Miskonsepsi yang Sering Terjadi

Anggapan keliru	Pemahaman yang tepat
“Indeks array dimulai dari 1.”	Pada Python dan banyak bahasa lain, indeks dimulai dari **0**.
“Stack dan queue sama saja.”	Berbeda arah pengambilan: LIFO vs FIFO.
“pop() mengambil data paling awal.”	Pada stack, `pop()` mengambil data **paling akhir** (puncak).
“dequeue mengambil data terakhir.”	`dequeue()` mengambil data **terdepan** (paling lama menunggu).
“Pilih struktur data yang paling terkenal.”	Pilih sesuai **kebutuhan pengambilan data**.
“Array selalu paling baik.”	Array cepat dibaca lewat indeks tetapi kurang luwes untuk sisip/hapus di tengah.
“Stack dan queue hanya teori.”	Keduanya dipakai sehari-hari: Undo, Back, antrean cetak.
“Menyimpan lebih banyak data tidak mengubah apa-apa.”	Pilihan struktur menentukan kecepatan dan kemudahan perubahan.

10. Rangkuman

- **Struktur data** adalah cara mengatur data. Cara menyimpan menentukan cara mengakses.
- **Array**: data berurutan dengan **indeks** (dimulai dari 0). Indeks terakhir = n − 1.
- **Stack (LIFO)**: `push` menambah di puncak, `pop` mengambil dari puncak. Cocok untuk Undo dan Back.
- **Queue (FIFO)**: `enqueue` menambah di belakang, `dequeue` mengambil dari depan. Cocok untuk antrean cetak dan antrean pelanggan.
- **Memilih struktur**: tanyakan data mana yang ingin diambil lebih dulu dan apakah perlu akses menurut posisi.
- **Linked list** (pengayaan) cepat untuk sisip/hapus tetapi lambat untuk akses menurut posisi.

11. Latihan Mandiri

1. Apa prinsip stack?
2. Apa prinsip queue?
3. Jika array `[A, B, C]`, elemen indeks 0 adalah?
4. Setelah `push(1)`, `push(2)`, `push(3)`, `pop()`, apa isi stack?
5. Setelah `enqueue(A)`, `enqueue(B)`, `dequeue()`, data apa yang keluar?
6. Pilih struktur data untuk antrean pelanggan dan beri alasan.
7. Array berisi 8 elemen. Berapa indeks elemen terakhir? Berapa indeks elemen ke-5?
8. Mengapa stack cocok untuk fitur Undo? Jelaskan dengan satu contoh.
9. Telusuri: `push(7)`, `push(2)`, `pop()`, `push(9)`, `pop()`, `pop()` pada stack. Tuliskan urutan data yang keluar dan isi akhir stack.
10. Telusuri operasi yang sama dengan `enqueue` dan `dequeue` pada queue. Bandingkan hasilnya dengan nomor 9.
11. **Studi kasus:** sebuah aplikasi sekolah memiliki dua fitur: (a) riwayat halaman yang dibuka agar tombol “kembali” berfungsi, (b) antrean pendaftaran lomba yang diproses sesuai urutan daftar. Pilih struktur data untuk masing-masing dan beri alasan.

Kunci dan Pedoman Jawaban

1. LIFO: terakhir masuk, pertama keluar.
2. FIFO: pertama masuk, pertama keluar.
3. A.
4. [1, 2].
5. A.
6. Queue, karena FIFO: pelanggan yang datang pertama dilayani pertama.
7. Indeks terakhir = 7. Elemen ke-5 berada pada indeks 4.
8. Perubahan terakhir harus dibatalkan lebih dulu (LIFO). Contoh: setelah mengetik, menebalkan, lalu mengganti warna, Undo pertama membatalkan penggantian warna.

9. Stack: [7] → [7, 2] → pop mengeluarkan 2, [7] → [7, 9] → pop mengeluarkan 9, [7] → pop mengeluarkan 7, [] (kosong). Urutan keluar: 2, 9, 7. Isi akhir: kosong.
10. Queue: [7] → [7, 2] → dequeue mengeluarkan 7, [2] → [2, 9] → dequeue mengeluarkan 2, [9] → dequeue mengeluarkan 9, [] (kosong). Urutan keluar: 7, 2, 9. Hasilnya berbeda dari stack (2, 9, 7) walau operasinya sama.
11. (a) Stack, karena halaman terakhir dibuka adalah yang pertama dikembalikan (LIFO).
    (b) Queue, karena pendaftar pertama harus diproses pertama (FIFO).

12. Refleksi

1. Struktur data mana yang paling mudah kamu bayangkan, dan mana yang paling membingungkan?
2. Kegiatan sehari-harimu yang mana yang berperilaku seperti stack atau queue?
3. Kesalahan apa yang kamu temukan saat menelusuri operasi, dan bagaimana memperbaikinya?
4. Satu hal tentang struktur data yang ingin kamu gali lebih lanjut?

(Kamu tidak wajib menceritakan hal pribadi.)

Pertemuan berikutnya: **Algoritma Standar: Pencarian (Searching) dan Pengurutan (Sorting).** Setelah tahu cara menyimpan data, kita belajar cara menemukan dan mengurutkannya.

13. Glosarium

Istilah	Arti
Struktur data	Cara mengatur dan menyimpan data dalam program
Array	Kumpulan data berurutan yang diakses lewat indeks
Indeks	Nomor posisi elemen dalam array (dimulai dari 0)
Elemen	Satu data di dalam struktur
Stack	Struktur data LIFO (terakhir masuk, pertama keluar)
Queue	Struktur data FIFO (pertama masuk, pertama keluar)
LIFO	*Last In, First Out*
FIFO	*First In, First Out*
Push / Pop	Menambah / mengambil data di puncak stack
Enqueue / Dequeue	Menambah di belakang / mengambil dari depan queue
Top / Front / Rear	Puncak stack / depan queue / belakang queue
Underflow	Kesalahan mengambil data dari struktur yang kosong
Linked list	Struktur data berupa rantai node yang saling menunjuk

Istilah	Arti
Node	Satu elemen pada linked list (data dan penunjuk)

Kartu Struktur Data

	Array	Stack	Queue
Aturan	Akses lewat indeks	LIFO	FIFO
Tambah	Di posisi tertentu/akhir	`push` di puncak	`enqueue` di belakang
Ambil	Lewat indeks	`pop` dari puncak	`dequeue` dari depan
Contoh	Daftar nilai	Undo, Back	Antrean cetak

Ingat: indeks mulai dari 0 · telusuri operasi langkah demi langkah · pilih struktur sesuai data mana yang ingin keluar lebih dulu.

Catatan Fasilitasi untuk Guru

- **Mulai dari peragaan fisik** (buku ke kardus untuk stack, bola ke pipa untuk queue), lalu jembatani ke diagram dan kode. Pastikan semua murid dapat melihat; sediakan label teks.
- **Tingkat 1:** puzzle array/stack dengan 4 data. **Tingkat 2:** stack dan queue dengan 6 data. **Tingkat 3:** kombinasi operasi dan penelusuran indeks.
- **Gunakan analogi akrab:** antrean kantin atau kasir, tumpukan buku atau piring, skenario game atau antrean yang dikenal murid.
- **Sediakan tabel "operasi–isi struktur–keluaran"** agar murid menulis hasil tiap langkah.
- **Kode Python bersifat opsional.** Murid yang belum nyaman dengan kode cukup memakai kartu dan diagram. Murid yang siap dapat mencoba contoh di bahan bacaan.
- **Tantangan Tingkat 3:** bandingkan array dan linked list dan rancang skenario penggunaan keduanya.
- **Jaga suasana aman:** peragaan boleh diganti kartu bagi murid yang tidak nyaman tampil.

Bahan Ajar Informatika: Algoritma Standar – Pencarian dan Pengurutan Dasar

Bubble Sort, Pencarian Linear, Binary Search, dan Diagram Alir (Flowchart) · Elemen Berpikir Komputasional · Fase E / Kelas X SMK · Pertemuan 12

Disusun oleh: Ikhsan Qadri, S.Pd.

Peta Belajar

Di Pertemuan 11 kamu belajar **menyimpan** data memakai array, stack, dan queue. Sekarang kita belajar **mengolah** data yang tersimpan: **mengurutkannya** dan **mencarinya**. Dua pekerjaan ini ada di balik papan peringkat, daftar kontak, katalog toko daring, hingga hasil pencarian. Setelah membaca dan berlatih, kamu diharapkan mampu:

1. Menjelaskan langkah **Bubble Sort**.
2. Membandingkan **pencarian linear** dan **Binary Search**.
3. Menjelaskan **mengapa Binary Search memerlukan data terurut**.
4. Menyajikan langkah algoritma dalam **diagram alir (flowchart)**.

Pertanyaan pemantik untuk dibawa selama membaca:

- Apakah Bubble Sort adalah cara tercepat?
- Bagaimana jika mencari data dengan prinsip membelah dua (Binary Search)?
- Mengapa data harus terurut sebelum Binary Search?
- Berapa langkah dibutuhkan untuk mengurutkan kartu?

> ***Pemahaman kunci:*** *algoritma yang tepat membuat pencarian dan pengurutan data lebih cepat. Pemilihan algoritma bergantung pada kondisi data.*

1. Mengapa Perlu Algoritma Pencarian dan Pengurutan?

Bayangkan kamu membuat **papan peringkat** dan **pencarian nama** pada aplikasi sekolah. Dengan 20 siswa, urutan dan pencarian bisa dilakukan sekilas. Dengan 20.000 siswa, caranya harus **sistematis dan efisien**, atau aplikasi terasa lambat.

Algoritma adalah urutan langkah yang jelas dan berhingga untuk menyelesaikan suatu masalah. Algoritma yang baik memenuhi ciri berikut:

- **Jelas:** tiap langkah tidak ambigu.
- **Berhingga:** akhirnya berhenti.
- **Punya masukan dan keluaran:** ada data awal dan hasil akhir.
- **Efektif:** langkahnya dapat dilakukan.

Mengukur Kecepatan dengan Menghitung Langkah

Untuk membandingkan algoritma, kita tidak mengukur detik (karena bergantung pada komputer), melainkan **jumlah langkah**, misalnya **jumlah perbandingan** dan **jumlah pertukaran**. Semakin sedikit langkah untuk data yang sama, semakin efisien algoritmanya. Itulah yang kamu catat pada LKPD 12.

2. Bubble Sort

Ide Dasar

Bubble Sort mengurutkan data dengan cara berulang-ulang **membandingkan dua elemen yang bersebelahan** dan **menukarnya bila urutannya salah**. Disebut "gelembung" karena elemen terbesar perlahan "mengapung" ke ujung kanan pada tiap putaran.

Langkah-langkah (urutan naik/*ascending*, dari kecil ke besar):

1. Mulai dari elemen paling kiri.
2. Bandingkan elemen dengan tetangga di sebelah kanannya.
3. Jika elemen kiri **lebih besar** dari elemen kanan, **tukar** keduanya.
4. Lanjutkan ke pasangan berikutnya sampai ujung kanan. Itu satu **putaran**.
5. Ulangi putaran dari awal. Jika dalam satu putaran **tidak ada pertukaran**, data sudah terurut dan proses berhenti.

Setelah putaran pertama, elemen **terbesar sudah berada di posisi paling kanan**. Setelah putaran kedua, dua elemen terbesar sudah di tempatnya, dan seterusnya.

Contoh 1: Mengurutkan 5, 2, 4 (soal diagnostik)

Putaran	Perbandingan	Susunan setelah putaran	Jumlah pertukaran
Awal	–	[5, 2, 4]	–
1	(5,2) tukar → [2,5,4]; (5,4) tukar → [2,4,5]	[2, 4, 5]	2
2	(2,4) tidak tukar; (4,5) tidak tukar	[2, 4, 5]	0

Putaran 2 tanpa pertukaran, sehingga proses berhenti. **Hasil akhir: 2, 4, 5.**

Contoh 2: Mengurutkan 7, 3, 5, 1 (soal asesmen akhir)

Putaran	Yang terjadi	Susunan setelah putaran	Jumlah pertukaran
Awal	–	[7, 3, 5, 1]	–

Putaran	Yang terjadi	Susunan setelah putaran	Jumlah pertukaran
1	(7,3) tukar → [3,7,5,1]; (7,5) tukar → [3,5,7,1]; (7,1) tukar → [3,5,1,7]	[3, 5, 1, 7]	3
2	(3,5) tidak; (5,1) tukar → [3,1,5,7]; (5,7) tidak	[3, 1, 5, 7]	1
3	(3,1) tukar → [1,3,5,7]; (3,5) tidak; (5,7) tidak	[1, 3, 5, 7]	1
4	tidak ada pertukaran	[1, 3, 5, 7]	0

Hasil akhir: 1, 3, 5, 7. Total pertukaran = 3 + 1 + 1 = **5**.

Fakta menarik: jumlah pertukaran sama dengan banyaknya pasangan yang "terbalik" pada data awal. Pada 7, 3, 5, 1 ada lima pasangan terbalik: (7,3), (7,5), (7,1), (3,1), (5,1).

Format Pencatatan LKPD

Tabel LKPD 12 memuat: **putaran, susunan kartu, dan jumlah pertukaran**. Isilah seperti tabel contoh di atas.

Berapa Langkah yang Dibutuhkan?

Untuk *n* data, pada kasus terburuk Bubble Sort membandingkan hingga **n(n – 1) ÷ 2** pasangan (bila ujung yang sudah terurut tidak diperiksa lagi). Banyaknya langkah tumbuh **jauh lebih cepat** daripada banyaknya data.

Banyak data (n)	Perbandingan maksimum n(n − 1) ÷ 2
4	6
5	10
8	28
12	66
16	120

Data dua kali lebih banyak bisa membutuhkan langkah **sekitar empat kali** lebih banyak. Itu sebabnya Bubble Sort lambat untuk data besar.

Apakah Bubble Sort Tercepat?

Tidak. Bubble Sort dipakai untuk belajar karena **sederhana dan mudah dibayangkan**, tetapi kurang efisien untuk data besar. Ada algoritma pengurutan lain yang jauh lebih cepat (misalnya

merge sort dan *quick sort*). Bahasa pemrograman biasanya menyediakan pengurutan bawaan yang sudah dioptimalkan, misalnya `sorted()` dan `list.sort()` di Python.

Cara Lain Mengurutkan Kartu

Saat mengurutkan kartu dengan metodemu sendiri, kamu mungkin memakai cara manusiawi seperti **menyisipkan** kartu ke posisi yang tepat (mirip *insertion sort*) atau **mencari yang terkecil lalu menaruhnya di depan** (mirip *selection sort*). Bandingkan jumlah langkahmu dengan Bubble Sort dan catat mana yang lebih sedikit langkah.

3. Pencarian Linear

Pencarian linear memeriksa elemen **satu per satu dari awal** sampai target ditemukan atau data habis.

Contoh: mencari 23 pada `[15, 40, 8, 23, 4]`.

Langkah	Elemen diperiksa	Hasil
1	15	bukan
2	40	bukan
3	8	bukan
4	23	**ditemukan** (indeks 3)

Kasus	Banyak elemen diperiksa
Terbaik	1 (target ada di elemen pertama)
Terburuk	**n** (target di akhir atau tidak ada)

Jawaban soal diagnostik: pada pencarian linear, kasus terburuk memeriksa **semua n elemen**.

Keunggulan: bekerja pada data **acak atau tidak terurut**, sederhana. **Kelemahan:** lambat pada data besar.

4. Binary Search (Pencarian Biner)

Ide Dasar: Membelah Dua

Pernahkah kamu bermain tebak angka 1–100? Strategi terbaik adalah menebak **angka tengah** (50), lalu mengetahui apakah angka sebenarnya lebih besar atau lebih kecil, sehingga **separuh kemungkinan langsung terbuang**. Itulah Binary Search.

Syarat: data harus terurut.

Langkah-langkah:

1. Tentukan batas bawah (`low`) dan batas atas (`high`) pencarian.
2. Hitung **posisi tengah** `mid`.
3. Bandingkan **target** dengan nilai di `mid`:
    - sama → **ditemukan**;
    - target **lebih kecil** → buang bagian kanan (geser `high` ke `mid - 1`);
    - target **lebih besar** → buang bagian kiri (geser `low` ke `mid + 1`).
4. Ulangi sampai ditemukan, atau sampai `low` melewati `high` (artinya **tidak ada**).

Contoh 1: Target Ada (Mencari 34)

Data terurut (indeks 0–7): `[3, 8, 15, 21, 27, 34, 40, 52]`

Langkah	low	high	mid	Nilai di mid	Keputusan
1	0	7	3	21	34 > 21, buang kiri → low = 4
2	4	7	5	34	**ditemukan** di indeks 5

Hanya **2 langkah**. Pencarian linear butuh 6 langkah untuk data yang sama.

Contoh 2: Target Tidak Ada (Mencari 10)

Langkah	low	high	mid	Nilai di mid	Keputusan
1	0	7	3	21	10 < 21, buang kanan → high = 2
2	0	2	1	8	10 > 8, buang kiri → low = 2
3	2	2	2	15	10 < 15, buang kanan → high = 1
–	2	1	–	–	low > high, **tidak ditemukan**

Mengapa Data Harus Terurut?

Binary Search membuang separuh data **berdasarkan satu perbandingan**. Keputusan itu hanya sah jika kita yakin bahwa semua nilai di sisi kiri lebih kecil dan semua nilai di sisi kanan lebih besar. **Jika data acak, keyakinan itu hilang** sehingga bagian yang dibuang bisa saja memuat target.

Bukti dengan contoh: cari angka 3 pada data **acak** `[40, 8, 52, 15, 3, 34, 21, 27]`. Angka 3 ada di indeks 4.

Langkah	low	high	mid	Nilai di mid	Keputusan
1	0	7	3	15	3 < 15, buang kanan → high = 2

Langkah	low	high	mid	Nilai di mid	Keputusan
2	0	2	1	8	3 < 8, buang kanan → high = 0
3	0	0	0	40	3 < 40, buang kanan → high = −1

Hasil: **"tidak ditemukan"**, padahal angka 3 ada. Bagian kanan yang dibuang di langkah pertama ternyata memuat angka 3. Jadi pada data acak Binary Search memberi **jawaban salah**, bukan sekadar lambat. Jawaban untuk LKPD: *Binary Search gagal pada data acak karena keputusan membuang setengah ruang cari tidak valid.*

Berapa Langkah Binary Search?

Setiap langkah membuang separuh data. Pada kasus terburuk, banyak langkah kira-kira $\lfloor \log_2 n \rfloor + 1$.

Banyak data (n)	Pencarian linear (terburuk)	Binary Search (terburuk)
8	8	4
16	16	5
1.024	1.024	11
1.000.000	1.000.000	20

Untuk satu juta data, Binary Search paling banyak **sekitar 20 langkah**. Keunggulannya makin besar saat data bertambah.

Perbandingan

Aspek	Pencarian linear	Binary Search
Syarat data	Tidak perlu terurut	**Harus terurut**
Cara kerja	Periksa satu per satu	Bandingkan nilai tengah, buang separuh
Terburuk (n data)	n langkah	sekitar $\log_2 n + 1$ langkah
Cocok untuk	Data kecil atau acak, sekali cari	Data besar, terurut, sering dicari

5. Memilih Algoritma Sesuai Kondisi Data

Pemilihan algoritma bergantung pada kondisi data.

Kondisi	Pilihan yang masuk akal	Alasan
Data sedikit, dicari sekali	Pencarian linear	Sederhana, mengurutkan dulu justru

Kondisi	Pilihan yang masuk akal	Alasan
		memakan langkah
Data besar, **sering dicari**, jarang berubah	**Urutkan sekali**, lalu Binary Search	Biaya pengurutan terbayar oleh banyak pencarian cepat
Data berubah terus (sering ditambah)	Pencarian linear, atau struktur data lain	Menjaga data tetap terurut juga butuh langkah
Data sudah terurut	Binary Search	Memanfaatkan urutan

Pada contoh aplikasi sekolah: papan peringkat perlu **diurutkan** menurut skor. Pencarian nama pada daftar siswa yang sudah diurutkan menurut nama dapat memakai **Binary Search**.

6. Menyajikan Algoritma: Pseudocode dan Flowchart

Pseudocode

Pseudocode adalah tulisan langkah algoritma dengan bahasa sehari-hari yang terstruktur, tidak terikat bahasa pemrograman.

Pseudocode Bubble Sort:

```
ULANGI
    ada_tukar = salah
    UNTUK j dari 0 sampai n-2
        JIKA A[j] > A[j+1] MAKA
            tukar A[j] dan A[j+1]
            ada_tukar = benar
SAMPAI ada_tukar = salah
```

Pseudocode Binary Search:

```
low = 0
high = n-1
SELAMA low <= high
    mid = (low + high) // 2
    JIKA A[mid] = target MAKA ditemukan di mid, selesai
    SELAIN ITU JIKA target < A[mid] MAKA high = mid - 1
    SELAIN ITU low = mid + 1
tidak ditemukan
```

Flowchart (Diagram Alir)

Flowchart menggambarkan langkah algoritma dengan simbol dan panah, sehingga alurnya mudah diikuti sebelum menulis kode.

Simbol	Bentuk	Arti

Simbol	Bentuk	Arti
Terminator	Oval/kapsul	Mulai atau Selesai
Proses	Persegi panjang	Langkah pengolahan (misalnya menukar data)
Keputusan	Belah ketupat	Pertanyaan ya/tidak (misalnya A[j] > A[j+1]?)
Input/Output	Jajar genjang	Memasukkan atau menampilkan data
Alur	Panah	Arah langkah berikutnya

Aturan menggambar: satu titik **Mulai** dan minimal satu **Selesai**; arah alur jelas (biasanya atas ke bawah); setiap keputusan memiliki dua cabang berlabel **Ya** dan **Tidak**; tulis isi simbol singkat.

Isi flowchart Bubble Sort (urutan simbol yang perlu kamu gambar):

No	Simbol	Isi	Lanjut ke
1	Terminator	Mulai	2
2	Input/Output	Masukkan daftar A dengan n data	3
3	Proses	ada_tukar ← salah; j ← 0	4
4	Keputusan	j ≤ n − 2 ?	Ya → 5; Tidak → 8
5	Keputusan	A[j] > A[j+1] ?	Ya → 6; Tidak → 7
6	Proses	Tukar A[j] dan A[j+1]; ada_tukar ← benar	7
7	Proses	j ← j + 1	kembali ke 4
8	Keputusan	ada_tukar = benar ?	Ya → 3; Tidak → 9
9	Input/Output	Tampilkan A (sudah terurut)	10
10	Terminator	Selesai	–

Menyajikan flowchart **sebelum coding** membantu menemukan kekeliruan logika lebih awal.

Kode Python (Opsional)

```
def bubble_sort(a):
    a = a[:]                        # salinan agar data asli tetap
    ada_tukar = True
    while ada_tukar:
        ada_tukar = False
        for j in range(len(a) - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
```

```
                ada_tukar = True
    return a

def binary_search(a, target):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
        if a[mid] == target:
            return mid            # indeks ditemukan
        elif target < a[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1                     # tidak ditemukan

data = bubble_sort([7, 3, 5, 1])
print(data)                       # [1, 3, 5, 7]
print(binary_search(data, 5))     # 2
```

7. Tantangan Lanjutan (Tingkat 3)

Misi: bandingkan jumlah langkah pencarian linear dan biner untuk **16 data** serta tulis pseudocode sederhana.

1. Siapkan 16 angka **terurut**.
2. Pilih tiga target: satu di awal, satu di tengah, satu tidak ada.
3. Catat jumlah langkah untuk pencarian linear dan Binary Search pada tiap target.
4. Isi tabel perbandingan, lalu jelaskan hasilnya.
5. Tulis pseudocode salah satu algoritma dengan bahasamu sendiri.

Hasil yang diharapkan: untuk 16 data, pencarian linear dapat sampai 16 langkah, sedangkan Binary Search paling banyak **5 langkah**.

8. Miskonsepsi yang Sering Terjadi

Anggapan keliru	Pemahaman yang tepat
“Bubble Sort adalah yang tercepat.”	Bubble Sort sederhana tetapi lambat untuk data besar.
“Binary Search bisa dipakai pada data apa saja.”	Hanya pada data **terurut**.
“Pada data acak, Binary Search hanya lebih lambat.”	Hasilnya bisa **salah** (target ada tetapi dianggap tidak ada).
“Binary Search selalu lebih baik daripada	Untuk data kecil atau sekali cari, linear bisa

Anggapan keliru	Pemahaman yang tepat
linear.”	lebih praktis karena tidak perlu mengurutkan.
“Satu putaran Bubble Sort sudah terurut.”	Satu putaran hanya menempatkan elemen terbesar di ujung. Perlu beberapa putaran.
“Jumlah langkah sama dengan jumlah data.”	Tergantung algoritma dan kondisi data.
“Flowchart hanya hiasan.”	Flowchart membantu memeriksa logika sebelum coding.
“mid = (low + high) ÷ 2 selalu bilangan bulat.”	Dalam program dipakai pembagian bulat (`//`).

9. Rangkuman

- **Bubble Sort** membandingkan pasangan bersebelahan dan menukar bila salah, diulang sampai tidak ada pertukaran. Mudah dipahami, tetapi lambat untuk data besar.
- **Pencarian linear** memeriksa satu per satu. Kasus terburuk memeriksa **n** elemen. Tidak butuh data terurut.
- **Binary Search** membandingkan nilai tengah dan membuang separuh ruang cari. **Wajib data terurut**. Terburuk sekitar $\log_2 n + 1$ langkah.
- Pada data acak, Binary Search dapat **salah** karena keputusan membuang separuh tidak valid.
- Pemilihan algoritma bergantung pada **kondisi data** (ukuran, terurut atau tidak, seberapa sering dicari).
- **Flowchart** dan **pseudocode** menyajikan langkah algoritma secara sistematis sebelum coding.

10. Latihan Mandiri

1. Urutkan 5, 2, 4 dengan Bubble Sort. Tuliskan hasil akhirnya.
2. Apa syarat Binary Search?
3. Pada pencarian linear, kasus terburuk memeriksa berapa elemen dari n?
4. Urutkan 7, 3, 5, 1 memakai Bubble Sort.
5. Jelaskan satu langkah Binary Search.
6. Mengapa data harus terurut untuk Binary Search?
7. Urutkan 4, 1, 3, 2 dengan Bubble Sort. Tuliskan susunan tiap putaran dan jumlah pertukarannya.
8. Pada data terurut `[2, 6, 9, 14, 20, 25, 31]`, tuliskan langkah Binary Search untuk mencari 25.

9. Berapa kira-kira langkah maksimum Binary Search untuk 1.024 data? Bandingkan dengan pencarian linear.
10. Sebutkan satu simbol flowchart beserta artinya.
11. **Studi kasus:** aplikasi sekolah memiliki daftar 5.000 nama siswa yang jarang berubah dan sering dicari. Algoritma pencarian apa yang kamu pilih, apa yang perlu dilakukan sebelumnya, dan mengapa?

Kunci dan Pedoman Jawaban

1. 2, 4, 5.
2. Data harus terurut.
3. Semua n elemen.
4. 1, 3, 5, 7.
5. Bandingkan target dengan nilai tengah, lalu buang setengah ruang cari yang tidak mungkin memuat target.
6. Agar keputusan membuang setengah ruang cari valid. Pada data acak, bagian yang dibuang bisa saja memuat target.
7. Awal [4, 1, 3, 2]. Putaran 1: (4,1) tukar → [1,4,3,2]; (4,3) tukar → [1,3,4,2]; (4,2) tukar → [1,3,2,4], 3 pertukaran. Putaran 2: (1,3) tidak; (3,2) tukar → [1,2,3,4]; (3,4) tidak, 1 pertukaran. Putaran 3: tidak ada pertukaran, selesai. Hasil [1, 2, 3, 4], total 4 pertukaran.
8. low = 0, high = 6, mid = 3 (nilai 14): 25 > 14 → low = 4. mid = (4+6)//2 = 5 (nilai 25): ditemukan di indeks 5. Hanya 2 langkah.
9. Sekitar 11 langkah, sedangkan pencarian linear dapat sampai 1.024 langkah.
10. Contoh: belah ketupat = keputusan (ya/tidak); oval = mulai/selesai; persegi panjang = proses.
11. Urutkan daftar nama sekali (misalnya dengan pengurutan bawaan), lalu gunakan Binary Search. Karena data besar, jarang berubah, dan sering dicari, biaya pengurutan sekali terbayar oleh banyak pencarian cepat.

11. Refleksi

1. Bagian mana dari Bubble Sort yang paling mudah dan mana yang paling membingungkan?
2. Bagaimana caramu mengurutkan kartu sebelum belajar Bubble Sort, dan apakah lebih sedikit langkah?
3. Di mana kamu menemukan pencarian atau pengurutan dalam aplikasi yang kamu pakai?
4. Satu hal tentang algoritma yang ingin kamu gali lebih lanjut?

(Kamu tidak wajib menceritakan hal pribadi.)

Pertemuan berikutnya: **Penerapan Struktur Data Kompleks (Array Multi-dimensi dan Dictionary) pada Sistem Penyimpanan Objek.**

12. Glosarium

Istilah	Arti
Algoritma	Urutan langkah jelas dan berhingga untuk menyelesaikan masalah
Sorting (pengurutan)	Menyusun data menurut urutan tertentu
Searching (pencarian)	Menemukan data tertentu dalam kumpulan data
Bubble Sort	Pengurutan dengan menukar pasangan bersebelahan yang salah urutan
Pencarian linear	Memeriksa data satu per satu dari awal
Binary Search	Pencarian dengan membelah dua ruang cari pada data terurut
Ascending / Descending	Urutan naik / urutan turun
Putaran (iterasi/pass)	Satu kali penelusuran data
Pertukaran (swap)	Menukar posisi dua elemen
Pseudocode	Langkah algoritma dalam bahasa sehari-hari terstruktur
Flowchart	Diagram alir langkah algoritma
low / high / mid	Batas bawah, batas atas, dan posisi tengah ruang cari

Kartu Algoritma Dasar

	Bubble Sort	Pencarian linear	Binary Search
Tugas	Mengurutkan	Mencari	Mencari
Syarat data	Tidak ada	Tidak perlu terurut	**Harus terurut**
Ide	Tukar pasangan bersebelahan yang salah	Periksa satu per satu	Bandingkan tengah, buang separuh
Langkah (n data)	sampai $n(n-1)\div 2$ perbandingan	sampai n	sekitar $\log_2 n + 1$

Ingat: catat langkah satu per satu · uji dengan data kecil dulu · Binary Search hanya untuk data terurut.

Catatan Fasilitasi untuk Guru

- **Peragaan fisik:** 5 murid sukarela dengan angka acak di dada bisa memeragakan Bubble Sort. Murid yang tidak nyaman tampil dapat memakai kartu. Beri narasi langkah dan salinan tertulis.
- **Tingkat 1:** 5 kartu. **Tingkat 2:** 8 kartu. **Tingkat 3:** 12 kartu dengan pencatatan langkah dan prediksi.
- **Sediakan templat tabel iterasi** (putaran, susunan, pertukaran) serta kertas flowchart.
- **Tema kartu boleh disesuaikan** (skor, daftar nama kegiatan sekolah).
- **Minta murid memprediksi** jumlah langkah sebelum menghitung, lalu bandingkan.
- **Hargai metode yang berbeda** antarkelompok (bukan hanya Bubble Sort) dan tekankan perbandingan jumlah langkah.
- **Flowchart sebelum coding:** pastikan murid dapat menjelaskan alur sebelum menulis kode.
- **Tantangan Tingkat 3:** pseudocode dan perbandingan 16 data.

Bahan Ajar Informatika: Struktur Data Kompleks – Array Multi-dimensi dan Dictionary

Penerapan pada Sistem Penyimpanan Objek: Memetakan, Memanggil, dan Menguji Rancangan Data · Elemen Berpikir Komputasional · Fase E / Kelas X SMK · Pertemuan 13

Disusun oleh: Ikhsan Qadri, S.Pd.

Peta Belajar

Di Pertemuan 11 kamu mengenal array, stack, dan queue. Data nyata jarang sesederhana satu deretan angka. Data nilai siswa punya **baris dan kolom**, data aset sekolah punya **atribut berlabel** (nama, jumlah, lokasi). Hari ini kamu belajar dua struktur yang cocok untuk data semacam itu: **array multi-dimensi** (khususnya array 2D) dan **dictionary (key–value)**, lalu **menguji** apakah rancanganmu tahan terhadap perubahan. Setelah membaca dan berlatih, kamu diharapkan mampu:

1. Memetakan data bervolume besar ke **array 2D** atau format **Key–Value/Dictionary**.
2. Menjelaskan cara **memanggil dan mengubah** data berdasarkan **indeks** atau **key**.
3. **Menguji ketangguhan** desain data terhadap perubahan data mendadak.
4. Menyajikan **peta data** beserta alasan pemilihan strukturnya.

Pertanyaan pemantik untuk dibawa selama membaca:

- Struktur data apa yang rapi untuk melacak 500 siswa dengan nilai 10 mata pelajaran?
- Bagaimana memanggil atau mengubah satu data spesifik?
- Apa beda mengakses data lewat indeks dan lewat key?
- Apa yang terjadi jika data bertambah mendadak?

> ***Pemahaman kunci:*** *himpunan data besar perlu struktur yang rapi. Array multi-dimensi dan dictionary memudahkan penyimpanan dan pemanggilan data kompleks.*

1. Dari Banyak Variabel ke Struktur yang Rapi

Bayangkan kamu harus menyimpan nilai 500 siswa untuk 10 mata pelajaran, yaitu **5.000 nilai**. Kalau memakai satu variabel untuk satu nilai (`nilai_ani_mtk`, `nilai_ani_inf`, …) kamu butuh **5.000 variabel**. Mustahil dikelola: sulit dicari, sulit diubah, mudah salah ketik.

Struktur data yang tepat membuat semuanya tertata:

Cara menyimpan	Bayangannya	Masalah/kelebihan
Variabel tunggal banyak	Ribuan catatan tempel berserakan	Tidak bisa diolah bersama, mudah tercecer

Cara menyimpan	Bayangannya	Masalah/kelebihan
Array 2D	Tabel dengan baris dan kolom	Rapi untuk data seragam
Dictionary	Kartu berlabel (nama: nilai)	Data dipanggil lewat nama label
Gabungan	Lemari berlaci, tiap laci berisi kartu berlabel	Cocok untuk data besar dan kompleks

2. Array 2D (Matriks)

Array 2D adalah array yang elemennya adalah array lagi: data tersusun dalam **baris** dan **kolom** seperti tabel. Di Python, array 2D ditulis sebagai **list di dalam list**.

Contoh: Nilai 3 Siswa untuk 3 Mata Pelajaran

	Kolom 0 (Matematika)	Kolom 1 (Informatika)	Kolom 2 (Bahasa)
Baris 0 (siswa 1)	80	75	90
Baris 1 (siswa 2)	70	85	60
Baris 2 (siswa 3)	95	88	92

```
M = [
    [80, 75, 90],   # baris 0
    [70, 85, 60],   # baris 1
    [95, 88, 92],   # baris 2
]
print(M[1][2])      # 60  -> baris indeks 1, kolom indeks 2
M[1][2] = 65        # mengubah nilai tersebut
print(M[0][1])      # 75  -> baris indeks 0, kolom indeks 1
```

Cara Membaca `M[baris][kolom]`

- **`M[1][2]`** artinya elemen pada **baris indeks 1** dan **kolom indeks 2**. (Ingat, indeks mulai dari 0.)
- Untuk memanggil elemen baris 0 kolom 1, tulis **`M[0][1]`**.
- Urutannya selalu **baris dahulu, baru kolom**.

Mengolah Seluruh Baris atau Kolom

Untuk menghitung rata-rata nilai tiap siswa (tiap baris), gunakan pengulangan:

```
for baris in M:
    print(sum(baris) / len(baris))
```

Setelah `M[1][2]` diubah menjadi 65, rata-rata ketiga siswa adalah sekitar 81,67; 73,33; dan 91,67.

Kelebihan dan Keterbatasan Array 2D

Kelebihan	Keterbatasan
Rapi untuk data **seragam** (semua baris punya kolom yang sama)	Posisi bermakna hanya karena **kesepakatan**: kolom 2 berarti apa? Harus dicatat terpisah
Mudah diolah per baris atau per kolom	Menambah/menyisipkan kolom **menggeser indeks** sehingga kode lama bisa rusak
Akses lewat indeks cepat	Kurang cocok untuk data yang atributnya berbeda-beda antarobjek

3. Dictionary (Key–Value)

Dictionary menyimpan data berupa pasangan **key** (kunci/label) dan **value** (nilai). Data dipanggil lewat **key**-nya, bukan lewat nomor posisi.

- **Key** adalah **label unik** untuk memanggil nilai.
- **Value** adalah isi data (angka, teks, atau bahkan struktur lain).

```
aset = {"nama": "Proyektor", "jumlah": 3}
print(aset["nama"])         # Proyektor
aset["jumlah"] = 4          # mengubah nilai key "jumlah"
aset["lokasi"] = "Lab 1"    # menambah pasangan baru
print(aset)                 # {'nama': 'Proyektor', 'jumlah': 4,
'lokasi': 'Lab 1'}
```

Operasi Dasar

Operasi	Contoh
Memanggil nilai	`aset["nama"]`
Mengubah nilai	`aset["jumlah"] = 4`
Menambah pasangan	`aset["lokasi"] = "Lab 1"`
Menghapus pasangan	`del aset["lokasi"]`
Memeriksa key ada atau tidak	`"nama" in aset`
Memanggil dengan nilai cadangan	`aset.get("kondisi", "-")` (hasil "-" bila key tidak ada)

Aturan penting:

- **Key harus unik.** Jika kamu menulis key yang sama dua kali, nilai lama **tertimpa** (data hilang tanpa peringatan).

- Memanggil key yang **tidak ada** menyebabkan kesalahan (`KeyError`). Pakai `in` atau `.get()` bila ragu.
- Key biasanya berupa teks atau angka. Pilih key yang **bermakna dan konsisten** (misalnya selalu "jumlah", bukan kadang "Jumlah" kadang "jml").

Indeks vs Key: Apa Bedanya?

Aspek	Akses lewat **indeks** (array)	Akses lewat **key** (dictionary)
Cara memanggil	Nomor posisi: `M[1][2]`	Nama label: `aset["jumlah"]`
Arti data	Tersirat pada posisi (harus diingat)	**Tertulis jelas** pada label
Jika data diselipkan	Indeks **bergeser**	Key tetap sama
Cocok untuk	Data seragam, berurutan	Data **berlabel** yang dipanggil menurut nama
Risiko	Salah posisi (misalnya kolom tertukar)	Salah ketik key (`KeyError`)

Kapan dictionary lebih cocok daripada array biasa? Ketika data **berlabel** dan dipanggil menurut **nama**, atau ketika tiap objek punya atribut yang berbeda-beda.

4. Struktur Gabungan dan JSON

Data nyata sering membutuhkan **gabungan**. Dictionary dapat berisi list atau dictionary lain (*bersarang*), dan list dapat berisi dictionary.

Contoh 1: Daftar Dictionary (List of Dictionary)

```
daftar_aset = [
    {"kode": "A01", "nama": "Proyektor", "jumlah": 3, "lokasi": "Lab
1"},
    {"kode": "A02", "nama": "Laptop", "jumlah": 30, "lokasi": "Lab
1"},
    {"kode": "A03", "nama": "Router", "jumlah": 5, "lokasi": "Ruang
Server"},
]
print(daftar_aset[1]["jumlah"])   # 30
```

Urutan tetap ada (lewat indeks list), dan tiap objek punya atribut berlabel (lewat key).

Contoh 2: Dictionary Berkunci Kode (Dictionary of Dictionary)

```
aset = {
    "A01": {"nama": "Proyektor", "jumlah": 3, "lokasi": "Lab 1"},
    "A02": {"nama": "Laptop", "jumlah": 30, "lokasi": "Lab 1"},
```

```
    "A03": {"nama": "Router", "jumlah": 5, "lokasi": "Ruang Server"},
}
print(aset["A02"]["jumlah"])      # 30  -> langsung ke objek A02
aset["A02"]["jumlah"] = 28        # mengubah satu data spesifik
```

Pada contoh ini, memanggil satu data spesifik cukup **dua kunci**: kode objek lalu nama atribut. Tidak perlu menelusuri satu per satu.

Menjawab Pemantik: 500 Siswa × 10 Mata Pelajaran

Beberapa rancangan yang masuk akal:

Rancangan	Bentuk	Memanggil nilai Informatika siswa S002	Catatan
Array 2D 500 × 10	Baris = siswa, kolom = mapel	`M[1][3]` (setelah mencari baris S002 dan kolom Informatika)	Nama siswa dan nama mapel harus disimpan terpisah. Indeks tidak bermakna sendiri
List of dictionary	Satu dictionary per siswa	cari dulu dictionary berkode S002, lalu `["nilai"]["Informatika"]`	Mudah dibaca, tetapi mencari siswa tertentu harus menelusuri
Dictionary berkunci NIS	`data["S002"]`	`data["S002"]["nilai"]["Informatika"]`	**Paling langsung**: siswa dan mapel dipanggil lewat nama

```
data = {
    "S001": {"nama": "Ani",  "nilai": {"Matematika": 90,
"Informatika": 85}},
    "S002": {"nama": "Budi", "nilai": {"Matematika": 78,
"Informatika": 92}},
}
print(data["S002"]["nilai"]["Informatika"])   # 92
data["S001"]["nilai"]["Informatika"] = 88     # mengubah satu nilai
```

Sekilas tentang JSON

JSON adalah format teks untuk menyimpan dan menukar data terstruktur, sangat mirip dictionary: pasangan key–value yang boleh bersarang. Format ini banyak dipakai aplikasi dan layanan daring.

```
{
  "kode": "A02",
  "nama": "Laptop",
  "jumlah": 30,
  "lokasi": "Lab 1",
```

```
  "baik": true
}
```

Aturan singkat JSON: key dan teks memakai **tanda kutip dua**, dan nilai logika ditulis `true`/`false` (huruf kecil). Pada Python, dictionary dapat diubah menjadi JSON dan sebaliknya dengan modul `json`.

Perbandingan data banyak variabel vs format terstruktur: pada format terstruktur, **satu objek = satu bungkus** yang memuat semua atributnya. Menambah atribut baru tidak menuntut variabel baru dan data yang saling berkaitan tetap menyatu.

5. Memilih Struktur yang Tepat

Tanyakan hal berikut:

Pertanyaan	Jika ya	Pilihan
Apakah semua baris punya **kolom yang sama**, dan data berupa tabel angka?	Ya	**Array 2D**
Apakah data **berlabel** dan dipanggil menurut **nama**?	Ya	**Dictionary**
Apakah ada **banyak objek**, masing-masing punya beberapa atribut berlabel?	Ya	**Gabungan** (list of dictionary atau dictionary berkunci kode)
Apakah objek sering dicari lewat **kode unik**?	Ya	**Dictionary berkunci kode**
Apakah urutan baris penting (misalnya papan peringkat)?	Ya	**List** (atau list of dictionary)

Ringkasnya: **array 2D untuk data tabel seragam, dictionary untuk data berlabel, dan keduanya dapat digabung.**

6. Memetakan Dataset ke Struktur Data (Peta Data)

Peta data adalah rancangan yang menjelaskan **data apa disimpan di mana, dengan struktur apa, dan bagaimana dipanggil**. Langkah menyusunnya:

1. **Pahami dataset dan atributnya.** Objek apa yang disimpan? Atribut apa saja? Mana yang unik (kode)?
2. **Pilih struktur** (array 2D, dictionary, atau gabungan) sesuai pertanyaan di bagian 5.
3. **Gambar peta data** (tabel atau diagram) dan beri nama pada tiap bagian.
4. **Tuliskan alur penyimpanan dan pemanggilan** satu data spesifik.
5. **Uji** dengan skenario perubahan (bagian 7).
6. **Catat alasan** pemilihan dan kelemahan rancangan.

Contoh: Katalog Aset Sekolah (Data Ilustrasi)

Kode	Nama	Jumlah	Lokasi	Kondisi
A01	Proyektor	3	Lab 1	Baik
A02	Laptop	30	Lab 1	Baik
A03	Router	5	Ruang Server	Baik
A04	Kabel LAN	40	Gudang	Cukup

Total jumlah seluruh aset = 3 + 30 + 5 + 40 = 78.

Rancangan A: array 2D

```
M = [
    ["A01", "Proyektor", 3, "Lab 1", "Baik"],
    ["A02", "Laptop", 30, "Lab 1", "Baik"],
    ["A03", "Router", 5, "Ruang Server", "Baik"],
    ["A04", "Kabel LAN", 40, "Gudang", "Cukup"],
]
# Jumlah Laptop: baris 1, kolom 2
print(M[1][2])      # 30
```

Rancangan B: dictionary berkunci kode

```
aset = {
    "A01": {"nama": "Proyektor", "jumlah": 3, "lokasi": "Lab 1",
"kondisi": "Baik"},
    "A02": {"nama": "Laptop", "jumlah": 30, "lokasi": "Lab 1",
"kondisi": "Baik"},
    "A03": {"nama": "Router", "jumlah": 5, "lokasi": "Ruang Server",
"kondisi": "Baik"},
    "A04": {"nama": "Kabel LAN", "jumlah": 40, "lokasi": "Gudang",
"kondisi": "Cukup"},
}
# Jumlah Laptop: kode A02, atribut jumlah
print(aset["A02"]["jumlah"])    # 30
```

Format Tabel LKPD 13

Tabel temuan LKPD memuat: **atribut/data, struktur yang dipilih, cara memanggil (indeks/key), cara mengubah, dan alasan**. Contoh pengisian (memakai Rancangan B):

Atribut/data	Struktur yang dipilih	Cara memanggil (indeks/key)	Cara mengubah	Alasan
Kode objek	Key tingkat pertama	`aset["A02"]`	Tidak diubah (kode bersifat unik)	Kode unik cocok sebagai key

Atribut/data	Struktur yang dipilih	Cara memanggil (indeks/key)	Cara mengubah	Alasan
	pada dictionary			
Nama aset	Key `"nama"`	`aset["A02"]["nama"]`	`aset["A02"]["nama"] = "Laptop Siswa"`	Label jelas, tidak bergantung urutan
Jumlah	Key `"jumlah"`	`aset["A02"]["jumlah"]`	`aset["A02"]["jumlah"] = 28`	Sering diubah, dipanggil menurut nama
Lokasi	Key `"lokasi"`	`aset["A02"]["lokasi"]`	`aset["A02"]["lokasi"] = "Lab 2"`	Mudah dibaca
Seluruh tabel (sebagai pembanding)	Array 2D	`M[1][2]`	`M[1][2] = 28`	Rapi, tetapi arti kolom harus diingat

7. Menguji Ketangguhan Desain terhadap Perubahan Mendadak

Rancangan yang baik tidak hanya benar hari ini, tetapi **tahan terhadap perubahan**. Mengujinya berarti membayangkan skenario perubahan lalu memeriksa apa yang rusak.

Skenario Uji

Skenario perubahan	Array 2D (Rancangan A)	Dictionary berkunci kode (Rancangan B)
Atribut baru ditambahkan, misalnya “tahun beli”	Jika ditambah di **akhir** baris, aman. Jika **disisipkan di tengah**, indeks kolom bergeser sehingga `M[1][2]` bisa merujuk data yang salah	Aman: cukup menambah key `"tahun_beli"`
Objek baru ditambahkan	Tambah satu baris di akhir	Tambah satu pasangan kode baru
Objek dihapus	Baris bergeser, indeks objek lain berubah	Hanya objek itu hilang, kode lain tetap
Data bertambah 10 kali lipat	Ukuran naik. Mencari objek tertentu harus menelusuri baris satu per satu	Memanggil lewat kode tetap langsung
Dua objek diberi	Tidak ada peringatan, ada dua baris	Objek lama **tertimpa**, data

Skenario perubahan	Array 2D (Rancangan A)	Dictionary berkunci kode (Rancangan B)
kode yang sama	kembar	hilang tanpa peringatan
Salah ketik key atau salah indeks	Mengambil kolom yang salah tanpa error (hasil keliru)	`KeyError` muncul sehingga kesalahan cepat terlihat
Sebagian atribut kosong	Perlu sel kosong penanda agar indeks tidak bergeser	Cukup tidak menulis key, periksa dengan `in` atau `.get()`

Menjawab Pertanyaan LKPD

1. Mengapa struktur ini paling sesuai? Jelaskan dengan mengaitkan pada data: apakah data berlabel, apakah dipanggil lewat kode, seberapa sering berubah.

2. Apa yang terjadi jika data bertambah 10 kali lipat? Misalnya 500 siswa × 10 mapel (5.000 nilai) menjadi 50.000 nilai. Pada array 2D, mencari siswa tertentu dengan menelusuri baris bisa sampai 5.000 langkah (untuk 5.000 siswa). Pada dictionary berkunci NIS, memanggil satu siswa tetap hampir langsung karena dictionary dirancang untuk pemanggilan lewat key. Penggunaan memori naik pada kedua struktur.

3. Apa kelemahan rancanganmu? Setiap struktur punya kelemahan. Menyebutkannya justru menunjukkan rancangan yang diuji. Contoh kelemahan dictionary berkunci kode: key harus unik dan konsisten, data tidak punya urutan tampilan khusus, memori lebih besar daripada array sederhana, dan salah ketik key menimbulkan `KeyError`. Contoh kelemahan array 2D: arti kolom tidak tertulis dan perubahan struktur menggeser indeks.

Menguatkan Rancangan

- Tulis **kamus data** (daftar key atau kolom beserta artinya).
- **Validasi** isian: tipe data benar, rentang wajar, kode unik (ingat pembersihan data Pertemuan 7).
- Gunakan **nama key yang konsisten**.
- **Uji dengan data contoh** sebelum dipakai pada data sebenarnya.

8. Etika dan Privasi

- Gunakan **data anonim** atau data aset/aktivitas yang tidak memuat identitas pribadi.
- Data nilai atau kehadiran siswa adalah **data pribadi**. Simpan dan bagikan seperlunya.
- **Jujur** pada kelemahan rancangan dan hasil pengujian.
- Hargai **perbedaan rancangan** antarkelompok selama alasannya logis dan teruji.

9. Miskonsepsi yang Sering Terjadi

Anggapan keliru	Pemahaman yang tepat
"`M[1][2]` artinya baris 1 dan kolom 2 (dihitung dari 1)."	Indeks mulai dari 0, jadi baris **kedua** dan kolom **ketiga**.
"`M[kolom][baris]`."	Urutannya **baris dahulu**, baru kolom.
"Key dictionary boleh sama."	Key harus unik. Key sama menimpa nilai lama.
"Dictionary pasti lebih baik daripada array."	Tergantung data. Array 2D rapi untuk tabel seragam.
"Key dan indeks sama saja."	Indeks adalah nomor posisi, key adalah label bermakna.
"Struktur bagus tidak perlu diuji."	Rancangan perlu diuji terhadap perubahan data.
"Menambah data tidak mengubah apa-apa."	Pada array 2D, penyisipan dapat menggeser indeks.
"JSON sama persis dengan dictionary Python."	Mirip, tetapi JSON adalah **teks** dengan aturan penulisan sendiri (kutip dua, true/false huruf kecil).

10. Rangkuman

- **Array 2D** menyimpan data dalam baris dan kolom, diakses dengan `M[baris][kolom]` (indeks mulai dari 0).
- **Dictionary** menyimpan pasangan **key–value**. Key adalah label unik untuk memanggil nilai.
- **Gabungan** (list of dictionary, dictionary berkunci kode) cocok untuk data besar dan kompleks.
- Pilih struktur menurut data: **tabel seragam → array 2D; data berlabel → dictionary; keduanya dapat digabung**.
- **Peta data**: pahami dataset → pilih struktur → gambar peta → tuliskan alur pemanggilan → uji → catat alasan dan kelemahan.
- **Uji ketangguhan** dengan skenario: atribut baru, objek baru/dihapus, data 10 kali lipat, key ganda, salah ketik.

11. Latihan Mandiri

1. Pada matriks M, apa arti `M[1][2]`?
2. Apa itu key pada dictionary?

3. Kapan dictionary lebih cocok daripada array biasa?
4. Tulis dictionary untuk aset: nama “Proyektor”, jumlah 3.
5. Bagaimana memanggil elemen baris 0 kolom 1 pada array 2D M?
6. Jelaskan satu alasan memilih dictionary.
7. Diberikan `M = [[2, 4, 6], [1, 3, 5]]`. Berapa nilai `M[1][0]`? Berapa nilai `M[0][2]`? Ubah elemen baris 1 kolom 2 menjadi 9, tulis hasil akhir M.
8. Diberikan `d = {"nama": "Ani", "nilai": 90}`. Tulis perintah untuk (a) memanggil nama, (b) mengubah nilai menjadi 95, (c) menambah key `"kelas"` berisi “X-1”.
9. Sebuah array 2D menyimpan data siswa dengan kolom: kode, nama, nilai. Sekolah menambah kolom “kelas” di antara “nama” dan “nilai”. Apa yang bisa rusak, dan bagaimana dictionary menghindarinya?
10. Rancang struktur untuk 8 data buku perpustakaan (kode, judul, jumlah, rak). Pilih array 2D atau dictionary, tuliskan satu contoh data, cara memanggil jumlah buku tertentu, dan alasannya.
11. **Studi kasus:** sekolah mencatat log aktivitas laboratorium. Satu catatan memuat tanggal, kode ruang, jumlah pengguna, dan catatan opsional yang tidak selalu ada. Struktur apa yang kamu sarankan dan mengapa?

Kunci dan Pedoman Jawaban

1. Elemen pada baris indeks 1 dan kolom indeks 2.
2. Label unik untuk memanggil nilai.
3. Saat data berlabel dan dipanggil menurut nama.
4. `{"nama": "Proyektor", "jumlah": 3}`.
5. `M[0][1]`.
6. Contoh: data dipanggil lewat key yang bermakna sehingga lebih jelas dan tidak bergantung pada urutan posisi.
7. `M[1][0]` = 1. `M[0][2]` = 6. Setelah mengubah `M[1][2] = 9`, M = `[[2, 4, 6], [1, 3, 9]]`.
8. (a) `d["nama"]`; (b) `d["nilai"] = 95`; (c) `d["kelas"] = "X-1"`.
9. Kolom “nilai” bergeser dari indeks 2 ke indeks 3, sehingga kode yang memakai `[2]` untuk nilai kini mengambil kelas, dan hasilnya keliru tanpa peringatan. Pada dictionary, key `"nilai"` tetap sama dan key baru `"kelas"` cukup ditambahkan.
10. Contoh jawaban: dictionary berkunci kode, misalnya `buku = {"B01": {"judul": "Jaringan Komputer", "jumlah": 4, "rak": "R2"}}`. Memanggil jumlah: `buku["B01"]["jumlah"]`. Alasan: buku dicari lewat kode unik, atribut berlabel. Array 2D juga dapat diterima bila arti kolom dicatat dan alasannya logis.
11. Dictionary (atau list of dictionary): tiap catatan berupa dictionary berkey `"tanggal"`, `"ruang"`, `"pengguna"`, dan `"catatan"` (key opsional ditulis hanya bila ada dan dipanggil dengan `.get()`). Array 2D kurang cocok karena kolom “catatan” tidak selalu terisi.

12. Refleksi

1. Rancangan data mana yang menurutmu paling rapi untuk datamu, dan mengapa?
2. Skenario perubahan apa yang paling membuat rancanganmu goyah?
3. Bagaimana cara kelompokmu menguji rancangan, dan apa yang kamu perbaiki?
4. Satu hal tentang struktur data yang ingin kamu gali lebih lanjut?

(Kamu tidak wajib menceritakan hal pribadi.)

Informasi pertemuan berikutnya akan disampaikan oleh guru.

13. Glosarium

Istilah	Arti
Array multi-dimensi	Array yang elemennya berupa array lagi (misalnya array 2D)
Array 2D (matriks)	Data dalam baris dan kolom, diakses `M[baris][kolom]`
Dictionary	Struktur data berpasangan key–value
Key	Label unik untuk memanggil nilai pada dictionary
Value	Isi data yang terhubung dengan sebuah key
Struktur bersarang (nested)	Struktur yang di dalamnya memuat struktur lain
List of dictionary	Daftar yang tiap elemennya berupa dictionary
JSON	Format teks data terstruktur berpasangan key–value
Peta data	Rancangan yang menunjukkan data, struktur, dan cara pemanggilannya
Ketangguhan (robustness)	Kemampuan rancangan bertahan terhadap perubahan
KeyError	Kesalahan karena memanggil key yang tidak ada
Kamus data	Daftar key/kolom beserta arti dan tipenya

Kartu Struktur Data Kompleks

	Array 2D	Dictionary	Gabungan
Bentuk	Baris × kolom	Pasangan key–value	Dictionary/list bersarang
Memanggil	`M[baris][kolo`	`d["key"]`	`data["S002"]["nilai"]["In`

	Array 2D	Dictionary	Gabungan
	`m]`		`formatika"]`
Cocok untuk	Tabel seragam	Data berlabel	Data besar dan kompleks
Waspadai	Indeks bergeser	Key ganda, salah ketik	Kerumitan, konsistensi key

Ingat: indeks mulai dari 0 · baris dahulu baru kolom · key harus unik · selalu uji dengan skenario perubahan.

Catatan Fasilitasi untuk Guru

- **Siapkan dataset anonim** bertema katalog aset sekolah, log kegiatan, atau layanan. Tingkat 1: 10 data dengan 3 atribut. Tingkat 2: 30 data dengan 5 atribut. Tingkat 3: 50+ data, dengan atribut yang tidak selalu lengkap.
- **Sediakan templat matriks dan kartu key–value** (kertas) agar murid dapat memetakan tanpa kode, serta contoh JSON sederhana di LMS.
- **Kode Python bersifat opsional.** Peta data cukup digambar sebagai tabel atau diagram. Hipotesis boleh berupa gambar atau tabel.
- **Latih murid menuliskan satu alur pemanggilan** (misalnya “buka objek A02, lalu ambil atribut jumlah”) sebelum mengisi kolom LKPD.
- **Dorong pengujian skenario** (atribut baru, data 10 kali lipat, key ganda) sebagai penalaran kritis. Kelemahan rancangan sebaiknya dihargai, bukan dihukum.
- **Beri kebebasan memilih tema data** dalam batas tujuan pembelajaran, dan hindari data pribadi siswa.
- **Tantangan Tingkat 3:** merancang struktur gabungan untuk 500 siswa × 10 mapel dan menulis pseudocode pemanggilannya.

Daftar Pustaka

Python Software Foundation. *Python Docs: Data Structures*. https://docs.python.org/3/tutorial/datastructures.html

freeCodeCamp. *Belajar algoritma dan struktur data dengan Python*. https://www.freecodecamp.org/news/learn-algorithms-and-data-structures-in-python/

Kemendikdasmen. (2025). *Keputusan Kepala BSKAP Nomor 046/H/KR/2025 tentang Capaian Pembelajaran (perubahan atas Nomor 032/H/KR/2024)*. Jakarta: BSKAP. https://guru.kemendikdasmen.go.id/kurikulum/referensi-penerapan/capaian-pembelajaran/